# 《数据结构与算法设计》课程设计计划及题目

## 一、目的

1.课程设计是一种全面的综合训练，是课堂理论教学的延续。

2.通过课程设计，让学生能深刻理解和灵活掌握课堂教学的内容，学会如何分析和解决复杂问题，进行复杂的程序设计训练，提高动手能力。

## 二、时间安排

1.7 月 6 日——7 月 10 日，集中进行课程设计相关内容讲解和上机训练。

2.7 月 10 日——9 月 6 日，自行安排上机试验，完成课程设计内容。

3.9 月 7 日下午 5 点前将课程设计内容按要求上传到 Canvas 系统中，主要包括：程序源代码，可执行程序，课程设计说明书。

## 三、总体要求

1. 提交内容

课程设计结果以电子文档的形式提交，需要提交的主要内容包括：

⑴《数据结构课程设计总结报告》：题目、软件功能、设计思想、逻辑结构与物理结构、开发平台、系统的运行结果分析说明、系统的安装、运行与操作文档；

⑵程序源代码；

⑶可实际运行的执行文件以及必须的相关配套文件（尽量提供完整，随作业一同提交）。

⑷录制视频：代码解读及核心测试。需录制一段 3 分钟左右、本人配音的屏幕录制视频，视频必须一镜到底，桌面系统时间必须全程连续可见。视频包括关键源码解读、核心测试展示两部分。

注意：以上各部分要提交电子文档材料。

2. 提交格式

所有提交文件压缩在一个压缩文件中，可以使用 rar 或 zip 格式。文件名清楚地写明：学号、姓名、专业等相关信息，格式为：

学号_姓名_专业.rar，如：240021_张三_计算机技术.rar

压缩包中目录为：

240021_张三_计算机技术_源代码（目录：目录内是源代码程序）

240021_张三_计算机技术_执行程序（目录：执行程序应包括涉及的配套文件）

240021_张三_计算机技术_课程设计总结报告.doc

240021_张三_计算机技术_课程设计.mp4

3. 选题要求

学生按学号最后一位数字，根据题目总览表，从题目中选择对应序号一题作为本人课程设计题目，题目 X 可自由选择。

4. 具体要求

学生在实现过程中，必须纯手工构建底层核心数据结构，严禁直接调用高级语言内置的封装集合库（如 Java 的 TreeMap、Python 的 networkx 等）。此外，所有题目均强制要

求开发高质量的功能交互或算法执行可视化面板，以全方位考查学生的数据结构设计能力、算法实现能力以及工程系统化思维。

## 四、题目总览表

| 编号 | 学号尾号 | 题目名称 | 核心应用领域 |
|---|---|---|---|
| 0 | 0、5 | 互联网视频直播高并发弹幕时间轴对齐缓存 | 泛互联网/流媒体 |
| 1 | 1、6 | 电商平台大规模大促广告位分时段动态竞价引擎 | 互联网广告/新零售 |
| 2 | 2、7 | 城市多模态公共交通智能接驳无缝导航 | 智慧城市/地图导航 |
| 3 | 3、8 | 智能网约车平台区域实时分布式派单与动态运力调度 | 共享经济/智能交通 |
| 4 | 4、9 | 电子病历（EMR）语义解析与急诊医疗资源动态抢占调度 | 智慧医疗/运筹调度 |
| X | 可任选 | 生物信息学大规模 DNA 序列片段高通量错配容错检索引擎 | 生物医药 / 基因计算 |

## 五、详细题目

### 题目 0：互联网视频直播高并发弹幕时间轴对齐缓存

**工程背景：**在大型网络直播（如顶级电竞赛事直播、跨年晚会）中，高并发弹幕在某些精彩瞬间会呈现爆发式增长（每秒数万条甚至数十万条）。弹幕服务器接收到来自前端的弹幕时，由于网络传输抖动、基站切换或客户端重发，弹幕到达服务器的时间戳往往是严重乱序的。为了保证所有观众在回放或同步观看时，弹幕能与视频画面的绝对时间轴精确对齐，服务器必须在内存中建立一个高效的滑动的时序缓冲区。该缓冲区需要对乱序到达的弹幕进行快速的原地插入排序，同时自动切除并释放超出滑动窗口的过期弹幕。此外，为了防止突发流量冲垮系统，必须严格限制单一时间窗口内的弹幕总量，触发基于权重的降级丢弃机制。

**设计与实现要求：**

1. **数据结构设计：**由学生完全手工实现一个支持快速局部查找与插入的时序表或带水平索引的双向循环链表。为了避免频繁高频申请/销毁节点带来开销，设计一个固定的对象池（Object Pool）。释放的弹幕节点不直接丢弃，而是回收到池中，新弹幕到达时优先从池中获取。
2. **核心算法设计：**实现动态滑动窗口更新算法，随着视频时间轴向前推进（模拟每秒前进 1 秒），系统需高效切除并回收窗口前（如 30 秒前）的过期弹幕节点；新弹幕到达时，根据其自带的视频时间戳，快速定位并插入到正确的位置，确保整条时间轴有序；当某一秒内的弹幕密度超过预设阈值（如每秒最多 500 条）时，触发降级：优先丢弃权重低的弹幕。

3. **设计要求：**编写一个单线程弹幕发射器，需成功处理以下边界情况：（1）故意制造大批延迟了 3-5 秒的“老弹幕”突发到达，测试系统乱序插入能力；（2）模拟某段时间没有弹幕输入，窗口依然正常滑动；（3）模拟当弹幕量超过承载上限时，自动启动丢弃策略。同时，以“带水平索引的双向循环链表”为目标方法，以传统“标准单链表”为基准方法。在不同数据规模下进行时间复杂度和空间复杂度性能测试，展示测试结果，绘制时空变化曲线图，并结合实验数据对两种方法的性能差异及原因进行分析。

4. **功能交互可视化：**无需实现复杂的底层指针动画，采用更直观的状态与数据可视化。为该系统搭建一个简易的图像化界面，展示弹幕飘过效果，并实时展示当前滑动窗口（例如当前在第 100 秒~130 秒之间）的整体结构；以柱状图或文字块颜色的深浅，实时展示这 30 个“时间槽”中每一个槽位的弹幕密度（条数）；当发生“过期节点回收”或“触发限流丢弃”时，在控制台高亮打印相关日志或触发界面闪烁提示。

**题目 1：电商平台大规模大促广告位分时段动态竞价引擎**

**工程背景：**在“双 11”或“618”等大型电商大促期间，平台的核心广告位（如 App 首页黄金 Banner）极其珍贵。广告主不再按整天购买，而是按照分钟甚至秒级区间进行极为精细的动态竞价。例如广告主 A 出价购买 14:00-14:30 的广告位，广告主 B 出价购买 14:15-15:00 的广告位。在极短时间内，会有海量类似的“区间出价更新”或“加价撤价”请求涌入。竞价引擎需要实时处理这些高频的区间修改，并随时应对系统的高并发查询——即任意特定时间点（或时间段）当前出价最高的广告主是谁，以便即时切播对应的广告快照。

**设计与实现要求：**

1. **数据结构设计：**学生需要手工实现一个高效的线段树，禁止直接调用系统现成的区间树第三方库。树的每个节点代表一个具体的时间区间（叶子节点代表最小时间单位，如 1 分钟），全天 24 小时共 1440 分钟（对应线段树的 1440 个叶子节点）。节点内部维护该区间的最高出价记录、当前胜出广告主 ID 链表以及用于性能优化的标记位。

2. **核心算法设计：**区间竞价更新，输入一个时间区间（如 [840, 870]）和出价，更新该区间内的最高价和赢家。若新出价高于原有出价则覆盖，低于则忽略。单点/区间最高价查询，查询某一分钟或某一段连续时间内，出价最高的广告主是谁，以便系统决定此时切播哪位广告主的广告。全额撤价，广告主可以撤销自己曾经在某个区间的出价，系统需要恢复该区间之前次高的有效出价。

3. **设计要求：**至少支持 4 个以上广告位。线段树的每个节点代表一个时间区间（如[0-1439] 表示全天，[840-870] 表示 14:00-14:30）。设计一个模拟生成器，生成至少 5000 条交错、重叠的竞价数据输入系统，验证线段树在大量区间交叉时的出价覆盖逻辑是否完全正确。

4. **功能交互可视化：**无需复杂的动画特效，重点展示数据结构在运行时的内部状态。**时段竞价看板**，用一根一维的“时间轴长条”（可以使用图形界面或者网页，甚至控制台字符画）展示全天 24 小时的广告位归属。不同广告主用不同颜色或代号表示。**竞价详情单点查询**，用户点击时间轴上的任意一个时间点（如 14:15），界面高亮显示该点的当前赢家和价格。**线段树路径溯源**，在查询或更新时，在控制台或文本框中实时打印出线段树被遍历的节点路径（例如：[Update] 访问节点 [0-1439] -> 访问子节点 [720-1439] -> 命中目标区间 [840-870]...），直观展示线段树的检索原理。

**题目 2：城市多模态公共交通智能接驳无缝导航**

**工程背景：**在现代智慧出行场景中，用户出行不再局限于单一交通工具，而是结合了“步行 + 地铁 + 公交车”的多模态联运。不同的交通工具其运行网络和特性完全不同：地铁准时且速度快，但站点固定；公交车覆盖广，但受路况影响可能拥堵；步行灵活但速度慢。更复杂的是，换乘存在“时间成本”（如地铁站内换乘需要步行 3 分钟，等公交车需要参考班次时刻表）。导航系统需要在这个多层复合的网络中为用户规划耗时最短、费用最省或换乘最少的路径。为了降低系统复杂都，题目中仅考虑“地铁 + 公交车”换乘。

**设计与实现要求：**

1. **数据结构设计：**学生需要手工实现图的存储结构，禁止直接调用现成的图论算法库。使用邻接表实现复合图结构。图中包含两类节点：地铁站和公交站。边结构非为 3 类：地铁边，连接两个地铁站，速度快（如 2 分钟/站），票价固定；公交边：连接两个公交站，速度慢（如 5 分钟/站），票价便宜；换乘步行边，连接位置邻近的地铁站与公交站，这类边的权重为固定的换乘惩罚项（例如：固定消耗时间 6 分钟，代表步行与等车时间）。

2. **核心算法设计：**实现基于最小堆优化的 Dijkstra 算法，手工实现最小堆（优先队列），用于加速 Dijkstra 算法，计算从起点站到终点站的最短路径。边的最终权重由时间与费用共同决定：W = 时间 + k * 费用。当用户选择“时间优先”时，设置 $k = 0$（纯算时间最短）；当用户选择“经济优先”时，调大$k$值（倾向于选择便宜的公交）。

3. **设计要求**：无需设计庞大的时刻表，仅需手动构建或用代码生成一个包含约 30-50 个站点（包含 2 条地铁线，3-5 条公交线，以及若干换乘连接点）的虚拟城市交通图。能处理多目标优化问题（支持用户选择“时间最短”或“费用最省”）。
4. **功能交互可视化**：基于图形界面开发一个虚拟城市交通仿真地图。用户在地图上任选起点和终点，并设置当前出发时间。系统以不同颜色的粗线在地图上描绘出导航路径，不同交通工具间切换时用特殊图标标注，并在文本框中打印出详细的文字乘车指南。

**题目 3：智能网约车平台区域实时分布式派单与动态运力调度**

**工程背景**：在网约车平台中，核心引擎每秒需要处理数万名乘客的呼叫订单，并将其分发给周边的空载司机。派单不仅要考虑距离（空间邻近度），还要考虑路况（时间邻近度）和司机的历史评分（服务质量）。更难的是，在早高峰时期，某区域内可能订单极多而空车极少（供需失衡），系统需要动态识别出这些热点区域，自动将周边其他区域的空载车辆引导（调度）过来，这需要一种高并发、多维度的时空数据处理框架。本题目要求手工实现一套简化的时空数据流缓存与撮合引擎，模拟网约车平台的派单与调度逻辑。

**设计与实现要求：**

1. **数据结构设计**：学生需要手工实现以下核心数据结构，禁止直接调用系统自带的并发集合类（如 ConcurrentHashMap）或第三方空间数据库。将虚拟城市地图（如 $1000 \times 1000$ 的正方形区域）均匀划分为 $10 \times 10$ 的网格矩阵。每个网格内部维护一个空闲司机链表。支持根据坐标在 O(1) 时间内定位网格，并快速增删司机。使用队列（Queue）存储当前未处理的乘客呼叫订单，遵循先来先服务（或按等待时间排序）的原则。当为一个订单寻找司机时，流式扫描当前网格及相邻网格的空闲司机，计算“撮合代价分数”，将其压入手工实现的最小堆中，堆顶即为最优匹配司机。撮合代价公式参考： Score = 距离 - 司机评分（分数越低越匹配）。
2. **核心算法设计**：网格化流式撮合算法，系统每隔固定时间（如模拟时间 1 秒）执行一次批量派单，从订单队列中取出订单，利用空间索引快速在周边网格中通过最小堆找出最合适的司机，匹配成功后将司机从空间索引中移除。供需失衡检测与动态调度，统计每个网格内的（订单数 - 空车数），当某个网格的差值超过阈值时，将其标记为“红色警报热点区”，系统自动将相邻网格中多余的空载司机“调度”（修改其坐标）到该热点区。状态机流转处理，

处理简单的业务边界，如：司机接单、订单因超时未匹配成功而被乘客取消等状态流转。

3. **设计要求：**设计一个模拟器，在 1000 × 1000 的坐标系中随机生成约 100 辆初始空闲出租车（带评分），并每秒随机产生 5-10 个呼叫订单。故意让某些区域的订单生成率极高，以测试供需失衡调度机制。

4. **功能交互可视化：**无需实现复杂的车辆平滑移动动画，重点展示时空网格的供需状态。**网格供需热力面板**，使用图形界面或简易画布，将地图划分为 100 × 100 的方格阵列。哪个格子的订单多，格子颜色变红；哪个格子的空车多，格子颜色变绿。能够直观看到因为“动态调度”机制，绿格子的司机逐渐向红格子转移的过程。**派单实时看板**，右侧或下方带有文本控制台，实时滚动打印撮合详情，例如：“[派单成功] 订单#1025 (位置: 450, 320) 已成功匹配 司机#088 (评分: 4.9, 距离: 120 米)，耗时 12ms”。**全局指标统计**，实时显示当前订单队列长度、全局派单成功率、系统总撮合耗时。

**题目 4：电子病历（EMR）语义解析与急诊医疗资源动态抢占调度**

**工程背景：**大型三甲医院的急诊部是一个典型的资源极度紧张、对时间极度敏感的综合控制中心。当患者到达急诊时，导医台首先录入其主诉症状文本（如“突发胸痛伴左上肢放射痛 1 小时”）。系统需要对该文本进行初步语义关键词解析，计算病情危重评分。随后，系统需将患者送入不同的急诊通道。由于 ICU 床位、急救医生、CT 检查设备等医疗资源严重有限，系统必须支持“优先级抢占”机制——当有特级濒危患者（如心梗、严重车祸）送达时，系统必须自动中断、挂起当前正在进行的普通急诊检查，将核心资源强行分配给最危险的患者，并在危机解除后恢复原患者的治疗。本题目要求手工实现一套基于症状解析的急诊排队与资源抢占仿真系统。

**设计与实现要求：**

1. **数据结构设计：**学生需要手工实现以下核心数据结构，禁止直接调用系统现成的标准库（如 PriorityQueue 或 TreeMap）。手工实现一棵标准字典树，用于存储和快速检索医学危险词（如“胸痛”、“咯血”、“昏迷”等），每个词带有不同的危险权重分。构建带索引的动态大顶堆，作为急诊患者的排队队列，堆顶永远是当前最危险的患者。为了支持患者病情突然恶化时能够动态提升其权重，必须维护一个位置映射表（用以在 O(log N)时间内精确定位节点并执行上浮操作）。构建资源状态表，管理医院的核心资源（如：3 间抢救室、2 台 CT 机）。每个资源槽位需记录当前占用者信息，以及被抢占挂起的“等待恢复者”上下文链表。

2. **核心算法设计**：语义分诊评分算法，输入一段患者主诉文本，通过字典树扫描并匹配出所有含有的医学关键词，累加其危险分，生成患者的初始“优先级权重”。引入时间步长模拟，随着在队列中等待的时间拉长，重症患者的病情可能会“自动恶化”（权重定时增加），系统需动态更新堆结构。当有更高权重的患者到达且资源全满时，强行中断当前权重最低的检查，将原患者移入“挂起等待队列”，资源换给新来的重症患者。当重症患者结束治疗释放资源时，系统自动恢复之前被挂起患者的治疗状态。

3. **设计要求**：构建一个包含 10-20 个典型急诊医学词汇的字典树；构造动态大顶堆，支持根据病人危急情况进行更新。构建 3 种 5 项以上医疗核心资源。模拟病人到达情况，实现“病情输入—病情标定—病人排队—资源抢占—恢复治疗”等过程。

4. **功能交互可视化**：无需复杂的底层指针重指向动画，重点展示资源流转与抢占过程。**语义解析高亮面板**，用户在界面输入一句话（如：“患者突发咯血并伴随昏迷”），界面能展示字典树匹配到的词汇（“咯血”、“昏迷”），并计算出最终得分。**急诊排队实时看板**，以动态列表或简易树状图展示当前大顶堆中病人的排队顺序，可以看到因病情恶化或新病人加入时，排队顺序的实时刷新。**医疗资源状态**，采用直观的网格或色块展示手术室和 CT 机的状态，绿色表示空闲，黄色表示普通患者治疗中，红色闪烁表示发生抢占（特危患者正在使用，旁边标记被其挂起的普通患者名字）。

**题目 X：生物信息学大规模 DNA 序列片段高通量错配容错检索引擎**

**工程背景**：在二代基因测序（NGS）中，测序仪会产生数以亿计的短序列片段（称为 Reads，长度通常为 50-150 个碱基，由 A、T、C、G 组成）。生物信息学家需要将这些短片段重新比对（Mapping）到长达数万碱基的参考基因组（Reference Genome）上。由于个体差异或测序仪误差，比对必须允许一定程度的“容错”（如允许最多 2 个碱基错配）。由于数据规模巨大，直接使用传统的字符串暴力匹配（如 KMP）完全不可行，必须在内存中对基因组构建高级查找索引。本题目要求实现一个容错检索引擎，通过建立高效的局部哈希索引，实现高通量的基因片段精确定位。

**设计与实现要求：**

1. **数据结构设计**：学生需要手工实现以下核心数据结构，禁止直接调用系统现成的 Hash 字典类（如 Java 的 HashMap）。手写拉链法哈希表。键（Key）：长度为 K 的固定碱基片段（称为 K-mer，例如 $K = 6$ 时，“ATCGAT” 是一个典型的 K-mer）。值（Value）：一个单链表（Linked List），里面记录了该

K-mer 在整个参考基因组中出现的所有起始下标位置。使用连续的字符数组存储长达 10KB 的参考基因组。

2. **核心算法设计：**K-mer 索引构建：用一个滑动窗口（步长为 1）纵览整个参考基因组，切出所有长度为 K 的短串，并将其对应的位置存入上述手写哈希表中。比对算法：当输入一个待查找的 Read（如长度为 30）时，先截取它的前 K 个字符作为“种子”；在哈希表中快速查出该“种子”在参考基因组中出现的所有可能位置（可能存在几十个候选点）；从这些候选位置开始，向后逐个字符对比整条 Read。计算两者的汉明距离（Hamming Distance，即不同字符的个数）。若不同的字符数≦ k（如 k=2），则认为比对成功，记录该匹配位置；若中途错配数超过 k，则立即剪枝（提前退出）。

3. **设计要求：**提供或用代码随机生成一段包含约 2,000 到 5,000 个碱基（仅含 A、T、C、G）的参考基因组序列。设计 50 条带突变（故意制造 1-2 处错配）的 Read 短串进行高通量检索测试。

4. **功能交互可视化：**无需展示抽象的后缀数组指针，重点展示基因序列的比对对齐效果。**基因组比对主沙盘**，在图形界面上，将长条形的参考基因组序列展示出来。**高亮比对结果**，当用户点击运行后，系统在沙盘上用粗线或特定底色框选出所有比对成功的区域。**错配字符染色（核心视觉）**：在比对成功的区域内，将那些由于“容错”而被允许的错配碱基用黄色或红色高亮标注出来，直观展示容错匹配的效果。